

Bases de Dados Relacionais e SQL I

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

Introdução

O que é uma Base de Dados?

Um conjunto estruturado de dados mantido num computador, especialmente um que seja acessível de várias formas.

- **Objetivo:** Armazenar, recuperar e gerir grandes quantidades de informação.
- **Evolução:**
 - **Flat files:** CSV/Texto (Rápido mas limitado).
 - **Hierárquico:** Estrutura em árvore (IBM IMS).
 - **Relacional:** Tabelas com relações (O padrão desde os anos 80).
 - **NoSQL:** Esquemas flexíveis para big data (MongoDB, Redis).

O que é um RDBMS?

Um **Sistema de Gestão de Bases de Dados Relacionais (RDBMS)** é a camada de software que gere as bases de dados relacionais.

- Fornece uma interface entre utilizadores/aplicações e os dados.
- Exemplos: **SQLite, PostgreSQL, MySQL, Oracle, Microsoft SQL Server.**
- Impõe o **Modelo Relacional**, a integridade dos dados e gere o acesso multutilizador.

RDBMS vs DB vs Modelo Relacional

- **Modelo Relacional:** A estrutura teórica (Codd, 1970) que utiliza tabelas (relações).
- **Base de Dados (DB):** A coleção real de dados (ex: o ficheiro `university.db`).
- **RDBMS:** O motor/software (ex: `sqlite3` ou `PostgreSQL`) que gere a DB usando o Modelo Relacional.

Conceito	Natureza	Responsabilidade
Modelo Relacional	Teoria	Define estruturas (Tabelas, Chaves).
Base de Dados (DB)	Dados	Os valores armazenados e metadados.
RDBMS	Software	Execução, Segurança, Integridade, ACID.

Vantagens do RDBMS

Um RDBMS garante a integridade dos dados através do ACID:

- **Atomicidade:** “Tudo ou nada.” Se qualquer parte de uma transação falhar, toda a transação é revertida.
- **Consistência:** A base de dados deve transitar de um estado válido para outro, seguindo todas as regras (restrições).

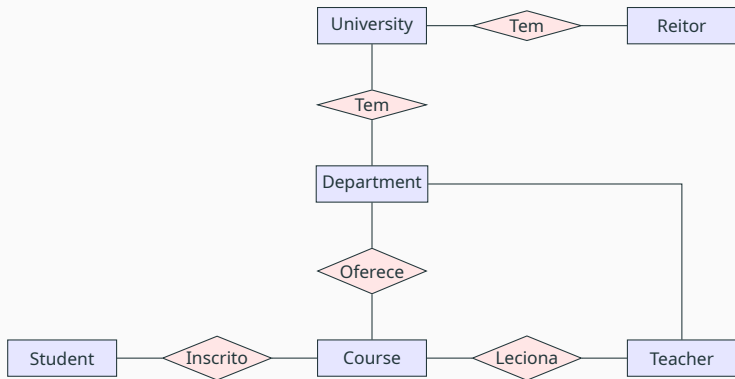
- **Isolamento:** Transações simultâneas não interferem; parecem correr sequencialmente.
- **Durabilidade:** Uma vez confirmada (committed), os dados são permanentes, sobrevivendo a falhas do sistema.

O Modelo Relacional

Para compreender estes conceitos, utilizaremos o cenário de uma **Base de Dados Universitária**:

- **University (Universidade)**: A instituição.
- **Rector (Reitor)**: O chefe da universidade (relação 1:1).
- **Department (Departamento)**: Unidades organizacionais (Engenharia, Artes).
- **Teacher (Professor)**: Pessoal afeto aos departamentos.
- **Course (Cadeira)**: Disciplinas oferecidas pelos departamentos.
- **Student (Aluno)**: Indivíduos inscritos nas cadeiras (relação N:M).

Diagrama Entidade-Relação (ER)



Proposto por Edgar F. Codd in 1970, organiza os dados em uma ou mais tabelas.

- **Tabela (Relação):** Uma coleção de elementos de dados organizados em linhas e colunas.
- **Linha (Tuplo/Registro):** Representa um item único na tabela.
- **Coluna (Atributo/Campo):** Representa uma propriedade específica dos itens.

Conceitos Base II

Coluna (Atributo)



ID	Nome	DeptID
1	Alice	10
2	Bob	10
3	Charlie	20

← Linha (Tuplo)

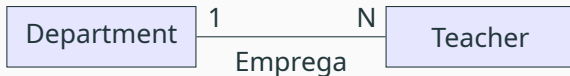
Relações em Bases de Dados I: 1:1

- **Um-para-Um (1:1):** Uma entidade está relacionada com exatamente uma outra.
- Exemplo: Uma **University** tem um **Reitor**.



Relações em Bases de Dados II: 1:N

- **Um-para-Muitos (1:N):** Uma entidade pode estar relacionada com muitas instâncias de outra.
- Exemplo: Um **Department** tem muitos **Teachers**.



Relações em Bases de Dados III: N:M

- **Muitos-para-Muitos (N:M):** Muitas instâncias de uma entidade relacionam-se com muitas de outra.
- Exemplo: Muitos **Students** inscrevem-se em muitas **Courses**.

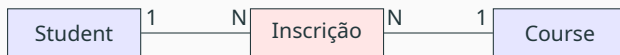


Tabela de Junção

SQL: Structured Query Language

Sublinguagens SQL (6 Modelos) I

O SQL divide-se em várias sublinguagens para diferentes propósitos:

1. **DQL (Data Query Language):** Para recuperar dados.
 - SELECT
2. **DML (Data Manipulation Language):** Para modificar dados.
 - INSERT, UPDATE, DELETE
3. **DDL (Data Definition Language):** Para definir o esquema.
 - CREATE, ALTER, DROP, TRUNCATE

4. **DCL (Data Control Language):** Para controlo de acessos/permisões.
 - GRANT, REVOKE
5. **TCL (Transaction Control Language):** Para gerir transações.
 - COMMIT, ROLLBACK, SAVEPOINT
6. **Administrativa/Metadados:** Para gerir o sistema ou inspecionar metadados.
 - DESCRIBE, EXPLAIN, PRAGMA (específico do SQLite)

Tipos comuns usados em RDBMS (Padrão e SQLite):

- **INT / INTEGER:** Números inteiros.
- **VARCHAR(n) / TEXT:** Cadeias de caracteres.
- **REAL / FLOAT / DOUBLE:** Números decimais.
- **DATE / DATETIME:** Valores temporais.
- **BOOLEAN:** Verdadeiro/Falso (frequentemente 0/1 no SQLite).
- **BLOB:** Dados binários (imagens, ficheiros).

Funções de Agregação SQL

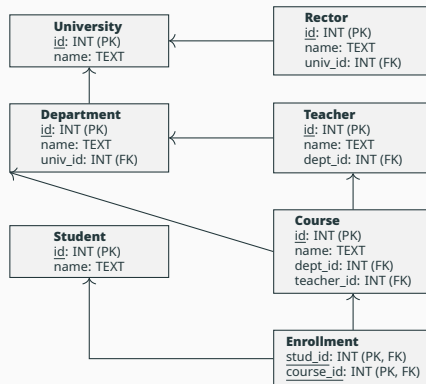
Funções que realizam um cálculo sobre um conjunto de valores:

- **COUNT():** Retorna o número de linhas.
- **SUM():** Retorna a soma total de uma coluna numérica.
- **AVG():** Retorna o valor médio.
- **MIN() / MAX():** Retorna o valor mínimo/máximo.

```
SELECT COUNT(*) FROM Student;
```

```
SELECT AVG(id) FROM Course; -- Cálculo de exemplo
```

Diagrama do Esquema Relacional



SQL de Fim-a-Fim: DDL (Esquema)

DDL: Criar Tabelas I

```
CREATE TABLE University (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL  
);
```

```
CREATE TABLE Rector (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,  
    univ_id INTEGER UNIQUE, FOREIGN KEY (univ_id) REFERENCES University  
);
```


DDL: Criar Tabelas II

```
CREATE TABLE Department (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,  
    univ_id INTEGER, FOREIGN KEY (univ_id) REFERENCES University(id)  
);
```

```
CREATE TABLE Teacher (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,  
    dept_id INTEGER, FOREIGN KEY (dept_id) REFERENCES Department(id)  
);
```

DDL: Criar Tabelas III

```
CREATE TABLE Course (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL,  
    dept_id INTEGER, teacher_id INTEGER,  
    FOREIGN KEY (dept_id) REFERENCES Department(id),  
    FOREIGN KEY (teacher_id) REFERENCES Teacher(id)  
);
```

```
CREATE TABLE Student (  
    id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL  
);
```

DDL: Criar Tabelas IV

```
CREATE TABLE Enrollment (  
    stud_id INTEGER, course_id INTEGER,  
    PRIMARY KEY (stud_id, course_id),  
    FOREIGN KEY (stud_id) REFERENCES Student(id),  
    FOREIGN KEY (course_id) REFERENCES Course(id)  
);
```

SQL de Fim-a-Fim: DML (Dados)

DML: Inserir Dados I

```
INSERT INTO University (name) VALUES ('Univ. Aveiro');
```

```
INSERT INTO Rector (name, univ_id) VALUES ('Prof. Paulo', 1);
```

```
INSERT INTO Department (name, univ_id) VALUES ('DETI', 1), ('DMat', 1);
```

```
INSERT INTO Teacher (name, dept_id) VALUES ('Dr. Smith', 1), ('Dr. Taylor', 1);
```

DML: Inserir Dados II

```
INSERT INTO Course (name, dept_id, teacher_id)  
VALUES ('Bases de Dados Relacionais', 1, 1), ('Álgebra Linear', 2, 2);
```

```
INSERT INTO Student (name) VALUES ('Alice'), ('Bob');
```

```
INSERT INTO Enrollment (stud_id, course_id)  
VALUES (1, 1), (1, 2), (2, 1);
```

DML: Atualizações e Eliminações

-- Modificar registos existentes

UPDATE Teacher **SET** name = 'Prof. Smith' **WHERE** id = 1;

-- Remover registos

DELETE FROM Enrollment **WHERE** stud_id = 2 **AND** course_id = 2;

SQL de Fim-a-Fim: DQL (Consultas)

DQL: Consultas Básicas e Filtragem

-- Selecionar colunas específicas com um pseudônimo (alias)

```
SELECT name AS NomeAluno FROM Student;
```

-- Filtragem com WHERE

```
SELECT * FROM Course WHERE dept_id = 1;
```

-- Funções de Agregação

```
SELECT COUNT(*) FROM Enrollment WHERE course_id = 1;
```

DQL: Consulta Complexa (Junções)

```
SELECT s.name AS Aluno, c.name AS Cadeira, t.name AS Professor,  
        d.name AS Dept, u.name AS Univ, r.name AS Reitor  
FROM Student s  
JOIN Enrollment e ON s.id = e.stud_id  
JOIN Course c ON e.course_id = c.id  
JOIN Teacher t ON c.teacher_id = t.id  
JOIN Department d ON t.dept_id = d.id  
JOIN University u ON d.univ_id = u.id  
JOIN Rector r ON u.id = r.univ_id;
```

Resultados Esperados I

Aluno	Cadeira	Professor
Alice	Bases de Dados Relacionais	Prof. Smith
Alice	Álgebra Linear	Dr. Taylor
Bob	Bases de Dados Relacionais	Prof. Smith

Resultados Esperados II

Dept	Univ	Reitor
DETI	Univ. Aveiro	Prof. Paulo
DMat	Univ. Aveiro	Prof. Paulo
DETI	Univ. Aveiro	Prof. Paulo

Outros Modelos SQL

Gere acessos e permissões (Concetual em alguns RDBMS):

```
-- Conceder acesso de leitura a um professor
GRANT SELECT ON Student TO 'user_professor';

-- Revogar permissões de eliminação aos alunos
REVOKE DELETE ON Course FROM 'role_aluno';
```

TCL: Transaction Control Language

Garante as propriedades ACID ao agrupar instruções:

```
BEGIN TRANSACTION;  
INSERT INTO Student (name) VALUES ('Eve');  
-- Se o sistema falhar aqui, Eve NÃO é adicionada permanentemente  
COMMIT; -- Guarda as alterações  
  
-- Ou reverter se for detetado um erro:  
-- ROLLBACK;
```

Comandos para gerir o próprio motor de base de dados:

```
-- MySQL / PostgreSQL:  
EXPLAIN SELECT * FROM Student;  
DESCRIBE Course;  
  
-- SQLite:  
PRAGMA foreign_keys = ON;  
PRAGMA table_info('Student');
```


Sumário

- **RDBMS:** Camada de software (SQLite) que impõe o **Modelo Relacional**.
- **ACID:** Vantagens fundamentais (Atomicidade, Consistência, Isolamento, Durabilidade).
- **Modelos SQL:** DQL, DML, DDL, DCL, TCL e Administrativo.
- **Estrutura:** Relações PK/FK visualizadas via diagramas ER e de Cardinalidade.